Quantum Machine Learning

Quick Recap

Day 13 introduced optimization and parameterized quantum circuits. In Quantum Machine Learning (QML), a classical optimizer similarly adjusts circuit parameters, but the goal is to learn patterns from data and make predictions.

1. What Is Quantum Machine Learning?

Quantum Machine Learning combines quantum computing with machine learning. Some data processing is carried out by quantum circuits; predictions, loss calculations, and parameter updates are often handled in a hybrid quantum-classical loop.

Everyday analogy: A flower classifier learns from examples labeled Species A or B. A QML classifier tries to learn the same relationship using a quantum circuit as part of the model.

2. Classical ML vs Quantum ML

Classical machine learning	Quantum machine learning
Data processed using classical numerical representations	Classical data encoded into quantum states
Computations use classical processors and numerical operations	Quantum gates and measurements process encoded states
Training commonly uses classical optimization	Training often combines quantum circuits and classical optimization

Important: QML is not automatically faster or more accurate. Classical CPUs, GPUs and TPUs remain highly effective; practical quantum advantage for general ML is not established.

3. The QML Workflow

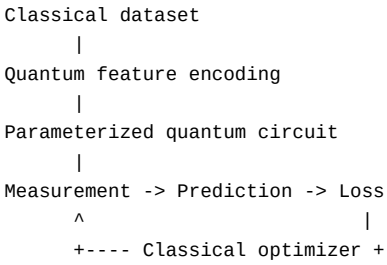


Figure 1. Hybrid QML training loop: the optimizer updates circuit parameters after evaluating prediction error.

4. Seven Steps of Training

1. Collect labeled data. 2. Encode features into qubits. 3. Apply a parameterized circuit. 4. Measure its output and make a prediction. 5. Compare with the label using a loss function. 6. Update gate parameters with a classical optimizer. 7. Repeat until performance is satisfactory.

5. Essential Mathematics

Quantum state: |psi> = alpha|0> + beta|1>. Here alpha and beta are probability amplitudes, and $|\text{alpha}|^2 + |\text{beta}|^2 = 1$ ensures measurement probabilities sum to one.

Trainable state: |psi(theta)> represents a quantum state produced by a circuit with adjustable parameters theta. A measurement of that state supplies information used to calculate a prediction.

Loss: L(y, y_hat) measures disagreement between the correct label y and the prediction y_hat. A classical optimizer proposes an updated parameter set theta_new to reduce loss. Lower training loss alone does not guarantee good results on unseen data.

6. Worked Example: Classifying Flowers

Suppose a flower has petal length 4.6 cm and petal width 1.5 cm, and its correct label is Species A. A quantum feature map encodes those measurements into a circuit. The trainable circuit initially predicts Species B, producing a loss. The classical optimizer adjusts the circuit parameters. After further iterations, the model may predict Species A more consistently.

Key lesson: The optimizer learns useful circuit parameters from examples; it does not memorize a guaranteed correct answer for every flower.

7. Qiskit: Variational Quantum Classifier

The workbook introduces a Variational Quantum Classifier (VQC) using ZZFeatureMap for data encoding, RealAmplitudes for trainable gates, COBYLA for classical optimization, and StatevectorSampler for circuit execution.

```
from qiskit.circuit.library import ZZFeatureMap, RealAmplitudes
from qiskit_machine_learning.algorithms import VQC
from qiskit_algorithms.optimizers import COBYLA
from qiskit.primitives import StatevectorSampler

feature_map = ZZFeatureMap(feature_dimension=2)
ansatz = RealAmplitudes(2, reps=2)
optimizer = COBYLA(maxiter=100)
sampler = StatevectorSampler()

vqc = VQC(sampler=sampler, feature_map=feature_map,
          ansatz=ansatz, optimizer=optimizer)
vqc.fit(training_features, training_labels)
predictions = vqc.predict(test_features)
print(predictions)
```

Before running: Define **training_features**, **training_labels** and **test_features** as compatible arrays. Install the relevant Qiskit packages; APIs can differ between releases. The code above is the workbook’s conceptual template, not a standalone runnable dataset example.

8. Where Researchers Explore QML

Area	Potential research use
Drug discovery	Learning molecular properties and screening candidates
Healthcare	Exploring classification of medical and imaging data
Finance	Fraud detection, risk analysis and forecasting
Cybersecurity	Network anomaly detection and malware classification
Computer vision	Specialized image classification and defect detection
NLP and science	Text classification, materials and scientific datasets

These are research directions and potential applications, not proof that QML currently outperforms classical systems in these domains.

9. Common Beginner Mistakes

“Quantum ML will replace classical ML.” No. Many tasks are better served by classical models today.

“Quantum hardware automatically improves AI.” No. Accuracy still depends on data quality, model choice, optimization and evaluation.

“A QML dataset must be quantum.” No. Most examples begin with classical data that must be encoded into quantum states.

“All training happens on a quantum processor.” No. Common variational QML approaches use classical optimizers to update parameters.

10. Current Limitations

QML faces data-encoding overhead, hardware noise, limited qubit counts, difficult optimization, and the need for strong classical benchmarks. Predictions can vary with parameter initialization, sampling and training conditions. Evaluation on held-out data matters more than a single successful run.

Summary: What to Remember

• QML combines quantum circuits with machine learning.
• Classical features usually require quantum encoding.
• Parameterized circuits are trained through repeated measurements and classical updates.
• VQC combines a feature map, ansatz, optimizer and sampler.
• Quantum advantage is a research question, not an automatic benefit.

Next: Day 15 — Grover’s Algorithm.